Reviewer Pack — Minimal Order of Quasi-Continuous Functions and Resolution P…

Entry 43e57a496ea9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-12 18:36:18 UTC

Minimal Order of Quasi-Continuous Functions and Resolution Proof Width

Entry ID: `43e57a496ea9` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-12 18:36:18 UTC

1. Conjecture

Field A (mathematical branch): Classical Mechanics - Quasi-Continuous Systems **Field B** (complexity object): Boolean Satisfiability Problem - Resolution Proof Complexity

Statement:

> For every Boolean satisfiability instance with n variables, the minimal order of a quasi-continuous function that can approximate the satisfaction relation is linearly correlated with its resolution proof width w(φ), such that O(n log n) ≤ w(φ).

Rationale (proposer's reasoning):

> Quasi-continuous functions in classical mechanics provide a bridge to continuous systems and their analysis. Their minimal order could potentially reveal a direct connection between the discrete structure of SAT instances and the complexity of resolving them, leading to new insights into resolution proof complexity.

Taxonomy category: `QuasiContinuity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ec15f9093385c882`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean order of quasi-continuous functions across all instances is within O(n log n) ± 10% of the resolution proof width w(φ), for at least 80% of seeds, and the highest order across any seed does not exceed O(n log n) + 20%.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.5s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction
from itertools import product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        literals = [f'x{i}' for i in range(n)]
        clauses = []
        for _ in range(2**n):
            clause = random.sample(literals, random.randint(1, n))
            clauses.append(' or '.join(clause))
        return ' and '.join(clauses)

    def evaluate_formula(formula, assignment):
        stack = []
        tokens = formula.split()
        for token in tokens:
            if token == 'and':
                b = stack.pop()
                a = stack.pop()
                stack.append(a + ' and ' + b)
            elif token == 'or':
                b = stack.pop()
                a = stack.pop()
                stack.append(a + ' or ' + b)
            else:
                stack.append(token)
        return stack[0]

    def truth_table(formula, n):
        variables = [f'x{i}' for i in range(n)]
        table = []
        for assignment in product([True, False], repeat=n):
            assignment_dict = {var: val for var, val in zip(variables, assignment)}
```

```
            result = evaluate_formula(formula, assignment_dict)
            table.append((assignment, result))
        return table

    def resolution(table):
        clauses = [set(clause.split(' or ')) for _, clause in table if clause == 'False']
        new_clauses = set()
        while True:
            found_new_clause = False
            for i in range(len(clauses)):
                for j in range(i + 1, len(clauses)):
                    for literal in clauses[i]:
                        if literal.startswith('not '):
                            neg_literal = literal[4:]
                            if neg_literal in clauses[j]:
                                new_clause = (clauses[i] - {literal}) | (clauses[j] - {neg_literal
                                if new_clause not in new_clauses:
                                    new_clauses.add(new_clause)
                                    found_new_clause = True
            if not found_new_clause:
                break
            clauses.update(new_clauses)
        return len(clauses)

    n_max = 40
    instances_tested = 30
    total_order = 0

    for _ in range(instances_tested):
        n = random.randint(5, n_max)
        formula = generate_formula(n)
        table = truth_table(formula, n)
        order = resolution(table)
        total_order += order

    mean_order = Fraction(total_order, instances_tested)

    return {
        "metric_name": "order",
        "metric_value": float(mean_order),
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)
```

```
mean_order = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not meet the pre-registered support conditions for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18572
2	preregistration	ollama_remote	glm4:latest	0	0	10066
3	novelty	ollama_remote	glm4:latest	0	0	9891
4	novelty	ollama_remote	glm4:latest	0	0	11079
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	56051
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11761
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14034
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10790
9	judge	ollama_remote	glm4:latest	0	0	54398

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 196641 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/43e57a496ea9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/43e57a496ea9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/43e57a496ea9.tar.gz` (if generated)